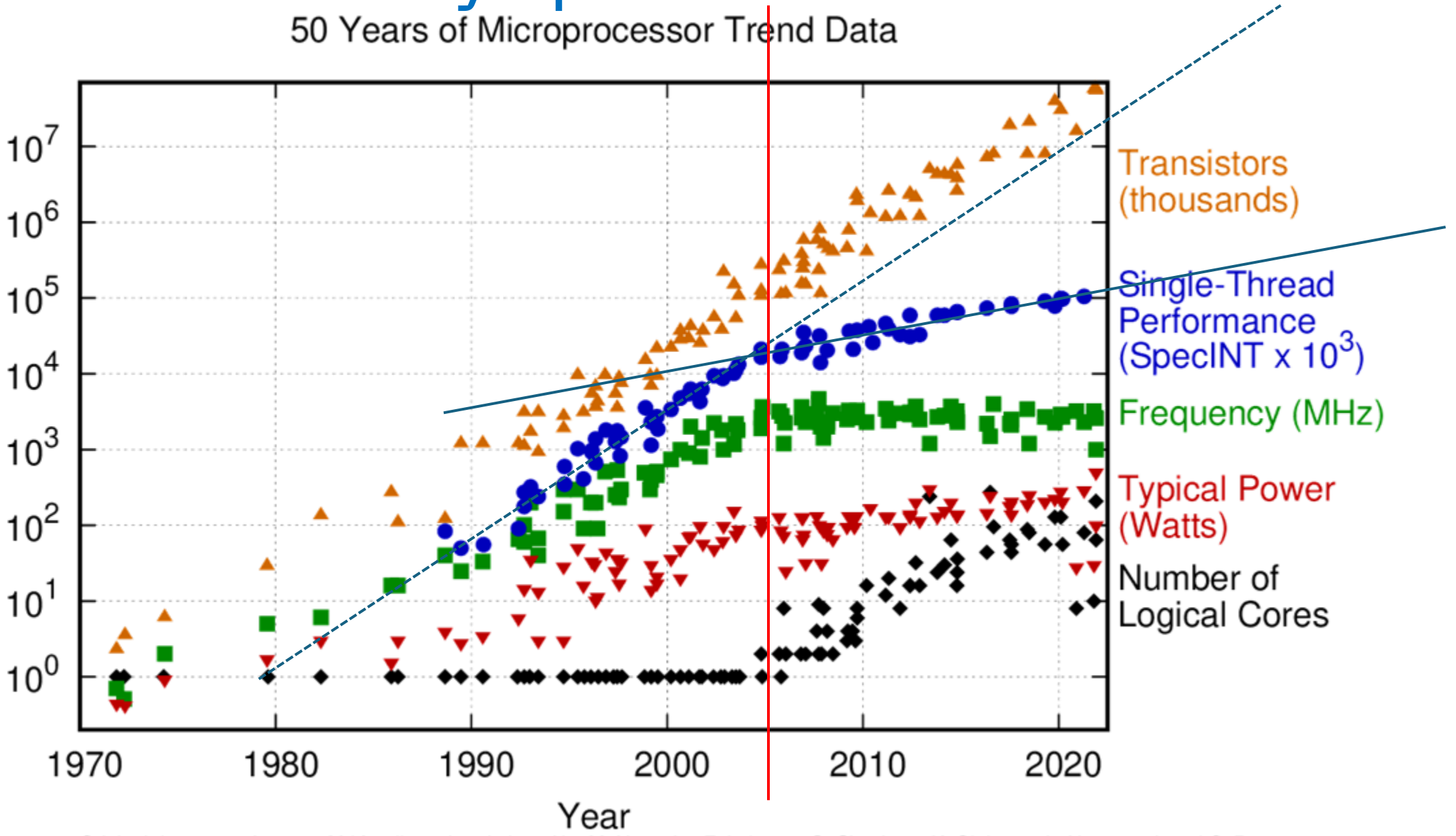
ASD – Software architecture

Parallel programming





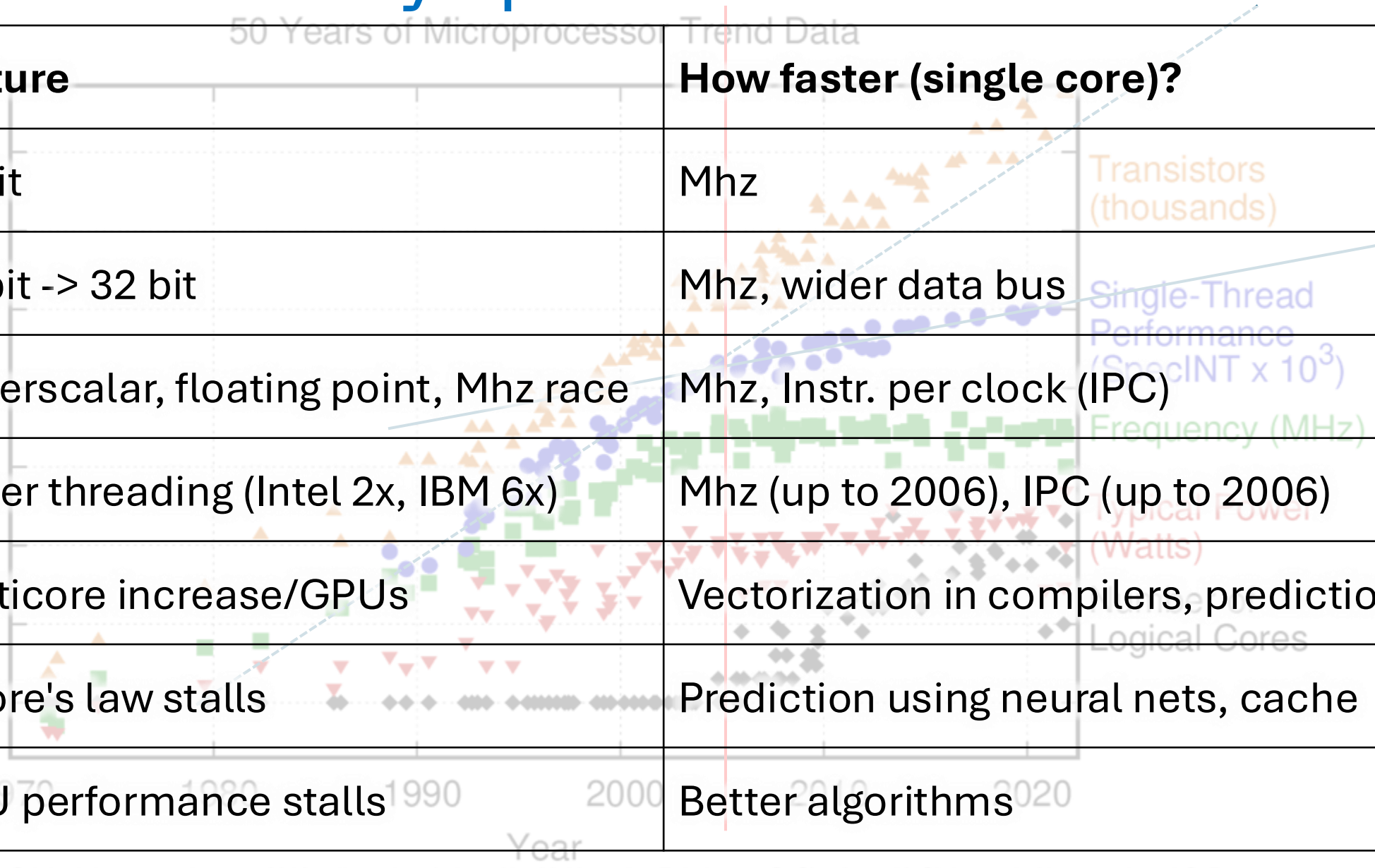
Why parallel?



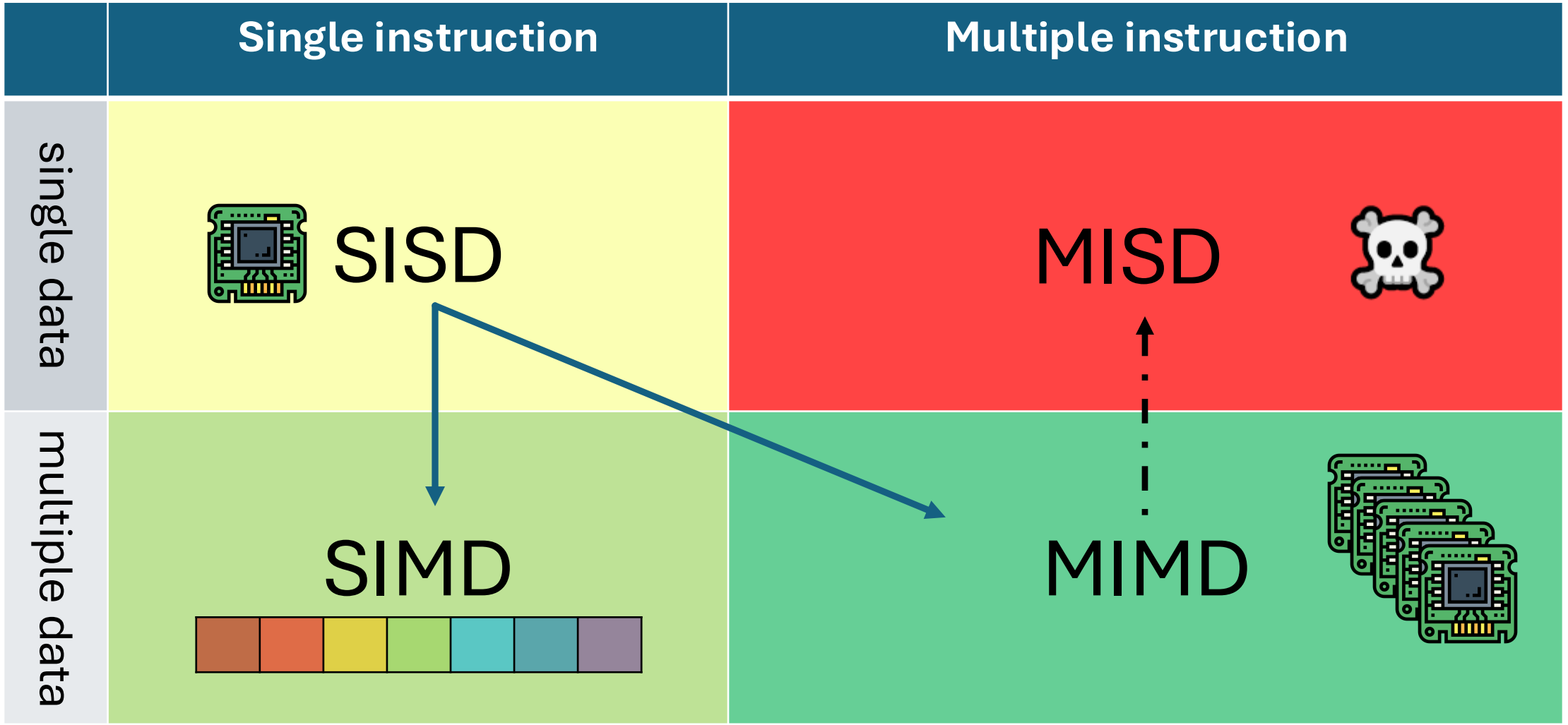
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

Why parallel?

year	feature	How faster (single core)?
1970	8 -bit	Mhz
1980	16 bit -> 32 bit	Mhz, wider data bus
1990	Superscalar, floating point, Mhz race	Mhz, Instr. per clock (IPC)
2000	Hyper threading (Intel 2x, IBM 6x)	Mhz (up to 2006), IPC (up to 2006)
2010	Multicore increase/GPUs	Vectorization in compilers, prediction
2020	Moore's law stalls	Prediction using neural nets, cache
2025	GPU performance stalls	Better algorithms



Flynn's taxonomy

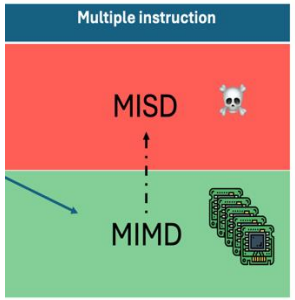


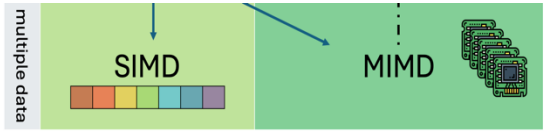
Parallel hierarchy…

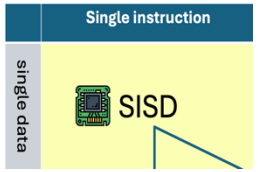
Task parallelism

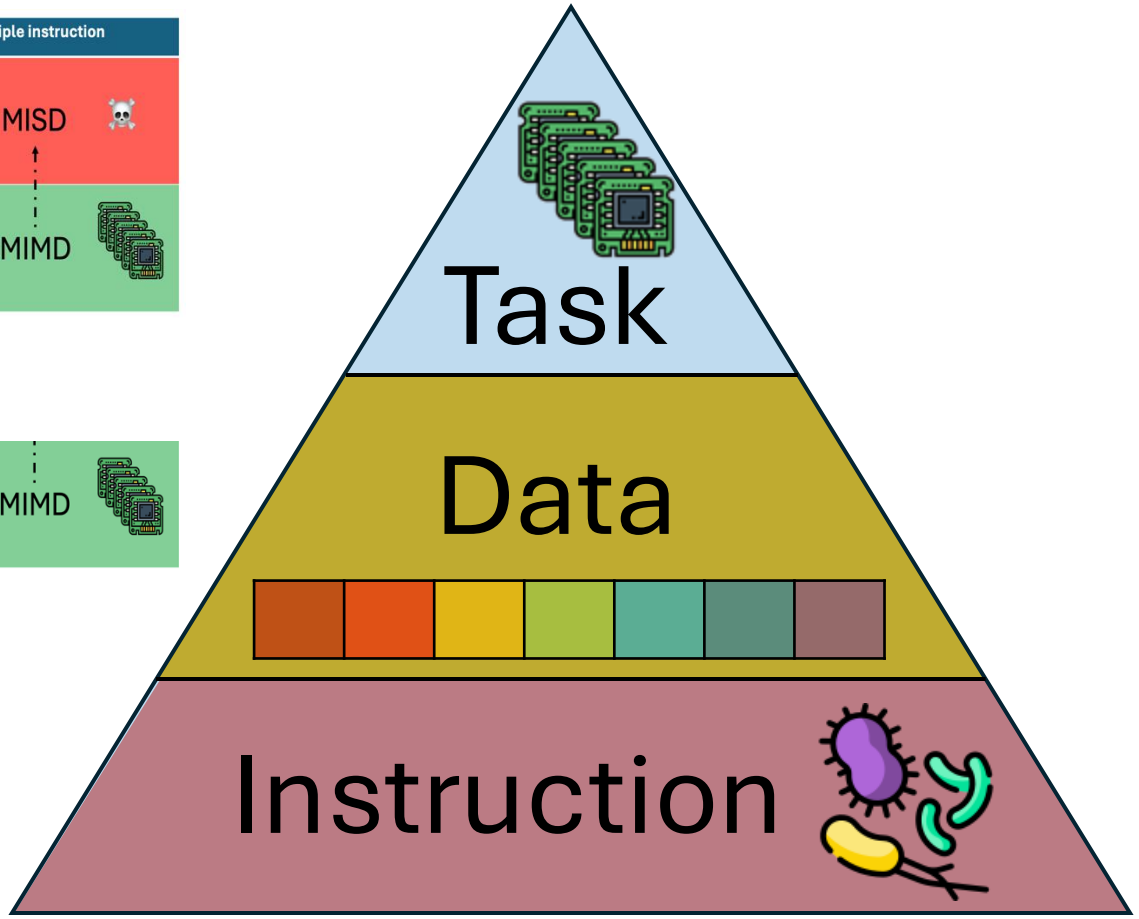
Data parallelism

Instruction parallelism









…Parallel hierarchy…

Task parallelism

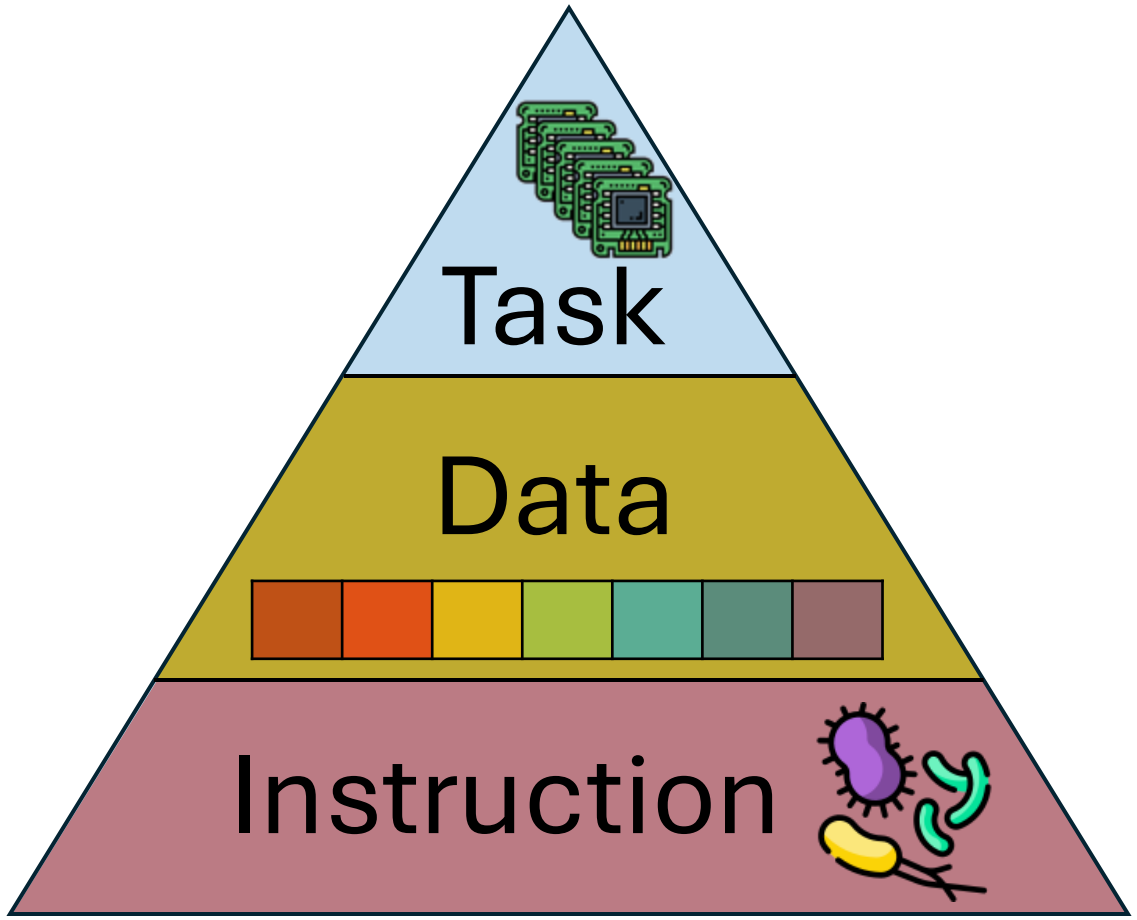
- concurrent ‘programs’
- data structures galore
- data races!

Data parallelism

- ‘Synchronized swimming’
- single task, multiple data
- multiple task, multiple data
- no racing

Instruction parallelism

- micro-optimizations:
  - pipelining
  - superscalar
- mostly automatic on CPU, manual on GPU

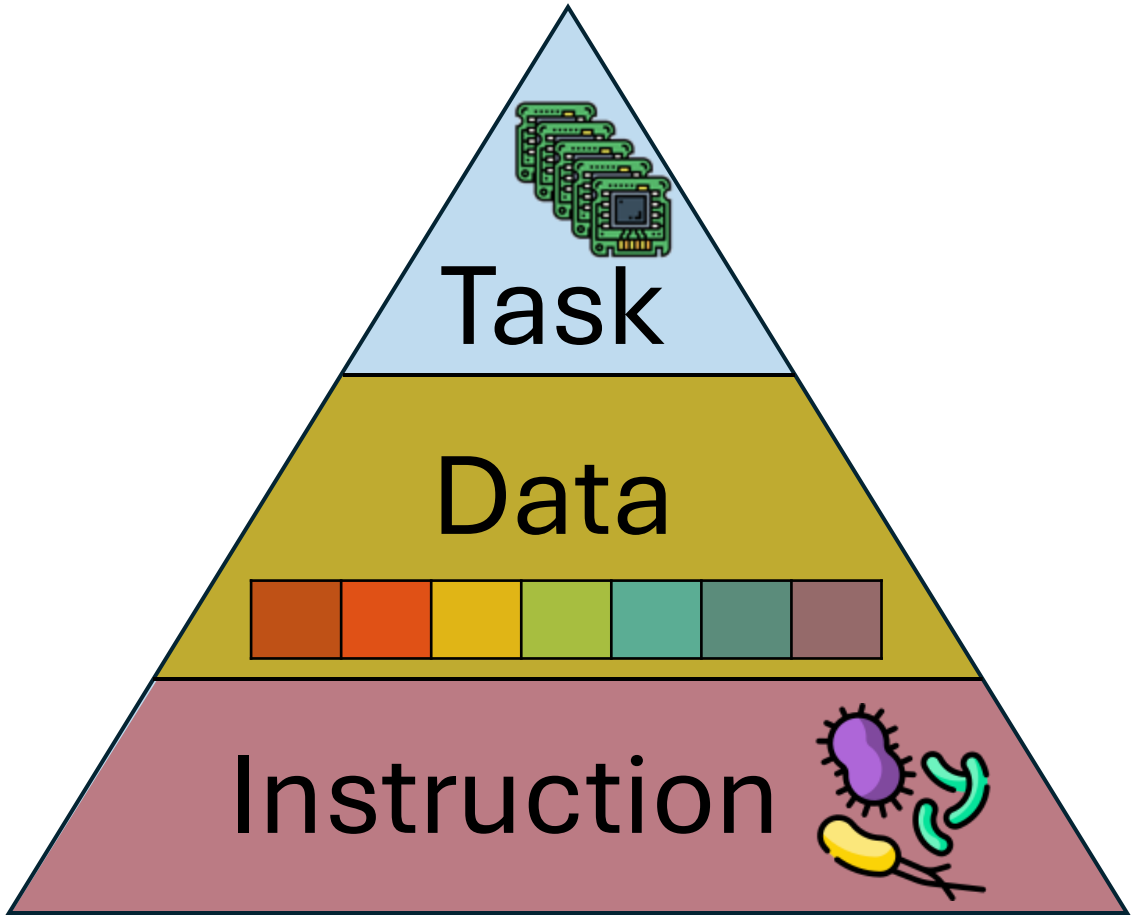


Parallel hierarchy…

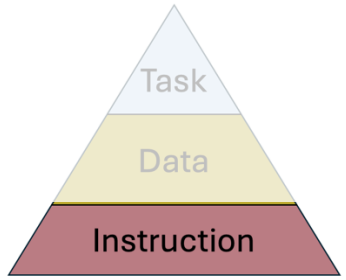
Task parallelism

Data parallelism

Instruction parallelism



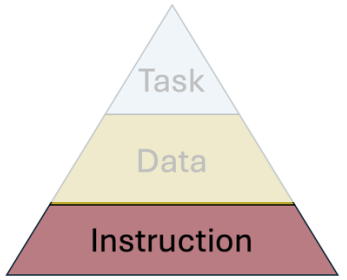
Instruction parallelism



Low level

- Bit parallelism (8 bit/16 bit/.../vector: 512 bits)
- Pipelining
  - Out of order (OoO) execution
  - Speculative execution
  - Branch prediction
  - Cache prediction
- Superscalar

Bit parallelism



Example:

```
A: int32, B: int32;
A += B;
```

On an 8-bit computer:

```
clc
mov a, [A+0]
mov b, [B+0]
adc a, b
mov [A+0], a
mov a, [A+1]
...repeat 4x
```

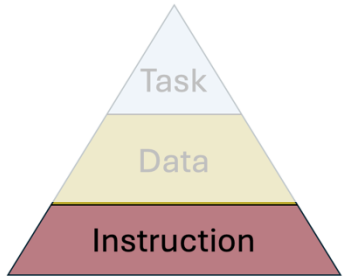
4x4+1 = 17 instr.
8 loads
4 stores
17 cycles*

On a 32-bit computer

```
mov a, [A]
add a, [B]
mov [A], a
```

3 instr.
2 loads
1 store
4 cycles

Bit parallelism



Example:

```
A, B: int32;
A += B;
```

On a 32-bit machine

```
mov a32, [A]
add a32, [B]
mov [A], a32
```

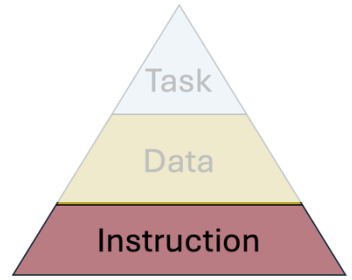
only works if (A, B >= 0)

```
X, Y: int64;
X = A[0] + (A[1] << 32);
Y = B[0] + (B[1] << 32);
A[0,1] = X + Y;
```

On a 64-bit machine

```
mov a64, [A]
add a64, [B]
mov [A], a64
```

Pipelining

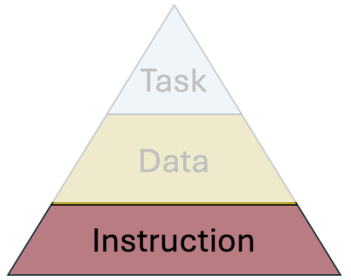


I1	IF	ID	EX	MEM	WB	...		
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB
I5					IF	ID	EX	MEM	WB

IF	Instruction fetch
ID	Instruction decode + data fetch
EX	Execute
MEM	Memory access (L1 Cache)
WB	Write back (L1 -> Mem)

Modern CPUs have ~20 stages in their pipeline

Pipeline stalls/flushes

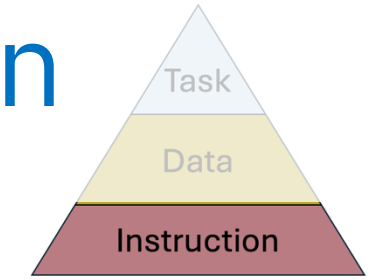


Assume B[0] == 2

A = B[0]	IF	ID	EX	MEM	WB	...				
isNeg = A < 0		IF	ID	stall	ID	EX				
if IsNeg:			IF	ID	stall	stall	EX			
A = -A				IF	ID	stall	stall	flush		
...					IF	ID	stall	flush	IF	ID	EX
...						IF	ID	flush	wait	IF	ID
							IF	flush	wait	wait	IF
								flush	wait	wait	wait

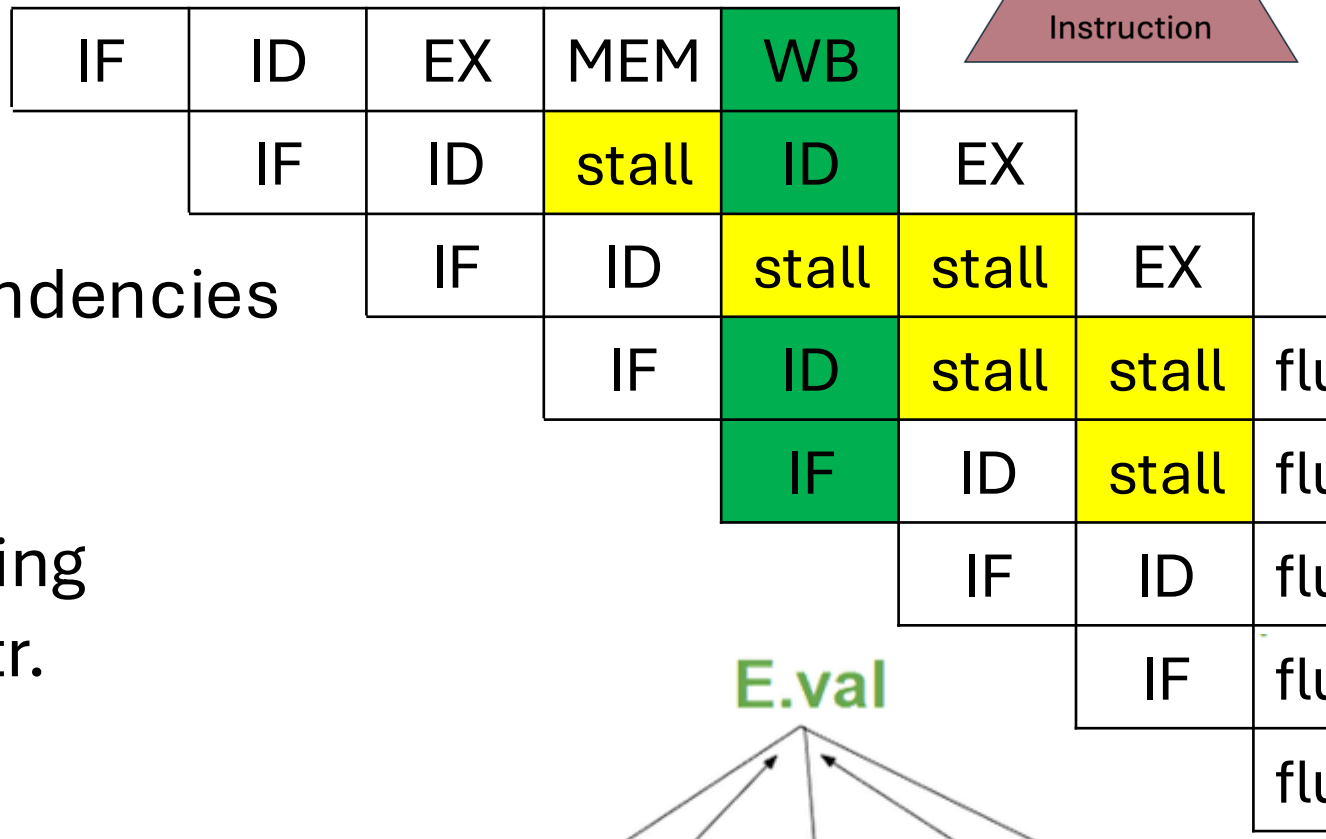
else: reload pipeline

OoO: Out of order execution



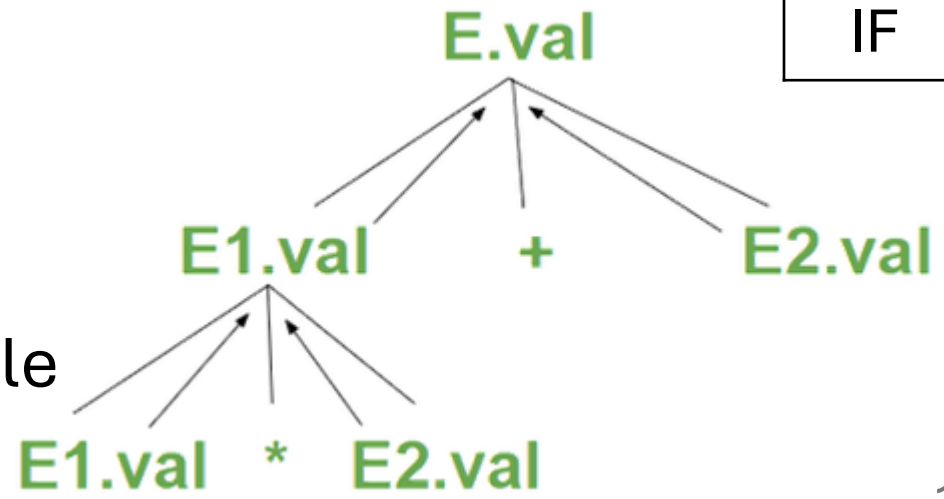
More complex pipeline

- CPU builds a dependency graph
  - Dependent/independent instr.
  - Latency delays between dependencies

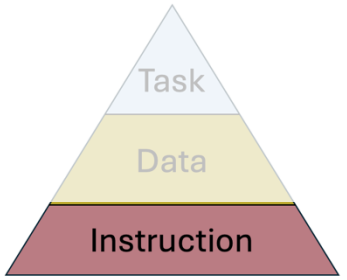


- Reorders instructions
  - Dependent instr. at ideal spacing
  - Fill gaps with independent instr.
  - Also called 'latency hiding'

- Needs redundant pipeline stages
  - CPU (IF/ID/EX/...) multiple instr. per cycle

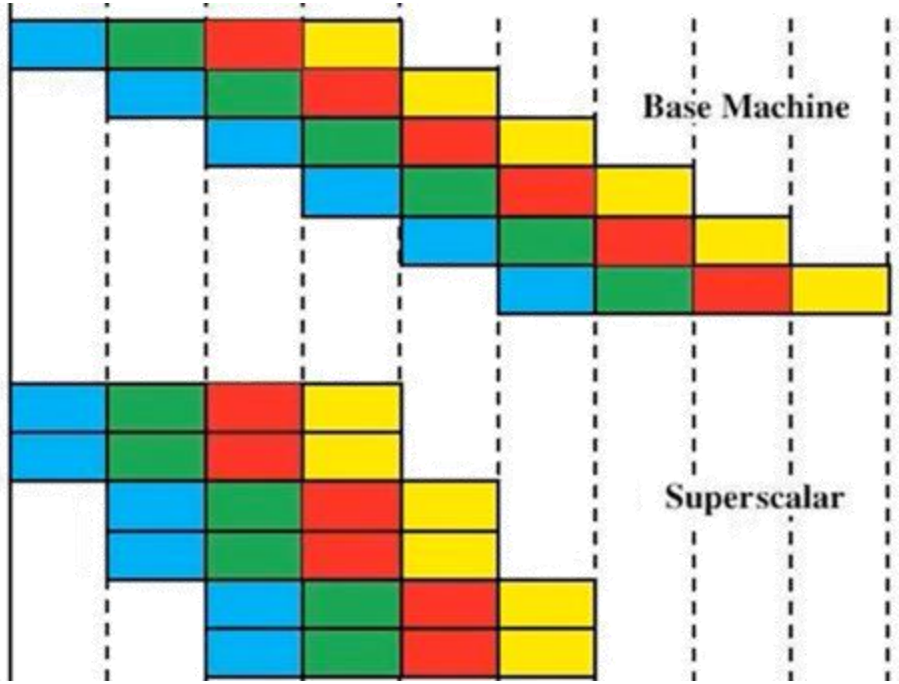


SS: Superscalar



Multiple instructions per cycle

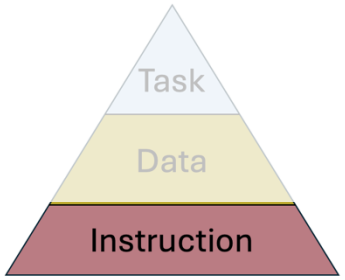
- Instr. cannot depend on each other
- Dependency graph deconflicts
- Almost always combined with OoO
  - *Original Intel Pentium is an exception*
- Needs duplicate execution units
- But uses a single redundant pipeline
  - allows flexibility in sceduling



Note: SS != multi-threading

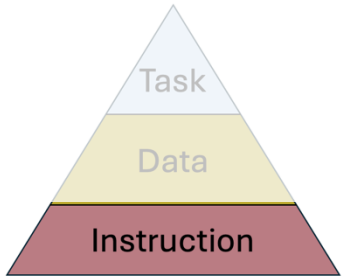
- SS: extract parallelism from 1 thread
- multi-threading: independent programs

SS/OoO



1. Instructions are fetched/decoded in order: [IF/ID]
2. Dependency graph build
3. Re-ordering happens
4. Instructions assigned to execution units
5. Executed out-of-order [data fetch + EX]
6. Results stored in output buffer [MEM] (Not a cache!)
7. output reordered to original program order
8. Output buffer moved to cache -> memory [WB]

OoO: Why CPU?



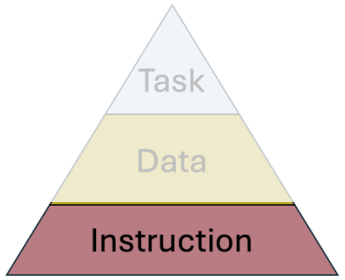
The **compiler** also reorders instructions, but

- it must err on the side of caution
  - no runtime knowledge — CPU does
  - cannot inspect data — CPU can
  - does not know flags
  - cannot rewind speculative execution

- compiler can only reorder once
  - the CPU can reorder differently given same instructions
    - if data or flags differ

- The CPU however,
  - only has limited reordering window (typically 1K instr.)
- compiler can do *whole program optimization*
- compiler can change instructions (e.g.: vectorization), CPU cannot

Speculative execution



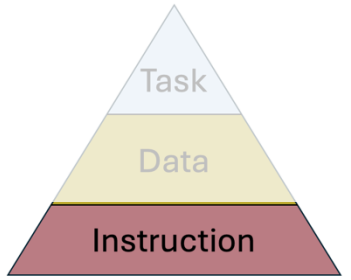
Assume B[0] == 2

cycle:	1	2	3	4	5	6	7	8	9	10	11
A = B[0]	IF	ID	EX	MEM	WB	...					
isNeg = A < 0		IF	ID	stall	ID	EX					
if isNeg:			IF	ID	stall	stall	EX				
A = -A				IF	ID	stall	stall	flush	predict isNeg: true		
...					IF	ID	stall	IF	ID	EX	MEM
...						IF	ID	flush	IF	ID	EX
							IF	flush	wait	IF	ID
								flush	wait	wait	IF

else: reload pipeline

Wrong prediction: 11 cycles, 5 EX

Speculative execution



Assume B[0] == 2

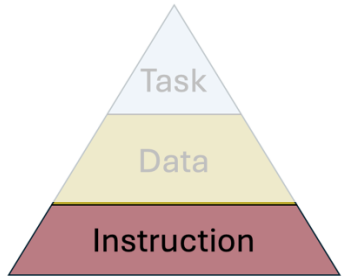
	1	2	3	4	5	6	7	8	9	10	11
A = B[0]	IF	ID	EX	MEM	WB	...					
isNeg = A < 0		IF	ID	stall	ID	EX					
if isNeg:			IF	ID	stall	stall	EX				
A = -A				IF	IF	ID	wait	EX			
...					wait	IF	ID	wait	EX	MEM	
...						wait	IF	ID	wait	EX	MEM
							wait	IF	ID	wait	EX
								wait	IF	ID	wait

predict isNeg: false

else: do other stuff

Correct prediction: 11 cycles, 7 EX

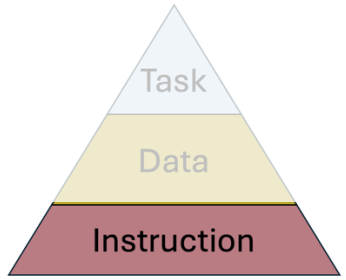
Speculative execution



With longer pipelines:

- Same penalty for stalls
- Much higher penalties for mispredictions
  - 5 stages: ~ 4 cycles
  - 20 stages: ~19 cycles
- Worse with OoO & superscalar
  - Dependency graph must be rebuilt
  - Multiple execution units may idle

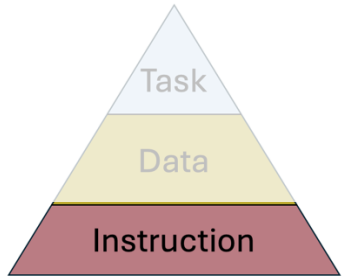
Speculative execution



Security implications…:

- CPU will execute speculatively up to output buffers
  - No stores
- May rewind, to undo instructions 👍
- But, **side channels** can still be observed
  - Loads cannot be undone
    - Data loaded into cache stays cached
    - Prediction counters in neural nets remain
  - This can be observed by timing delays in cache/jumps ⏱
    - Thus Boolean flags or bits can be deduced

Speculative execution



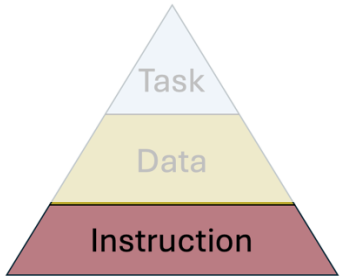
Security implications (cont.)

- Speculative reads do not cause exceptions
  - can read 'forbidden' data
  - this can be extracted using **side channel**

```
for(;;) { //trick CPU into assuming F == true
   if (F) { a = forbidden[i];
      if (a & 1) b = cache_me0[i]; //read bits
      if (a & 2) c = cache_me1[i];
```

- CPU will happily execute 100 speculative instr.
  - Modern CPU can undo up to a 1000 instructions

Speculative execution

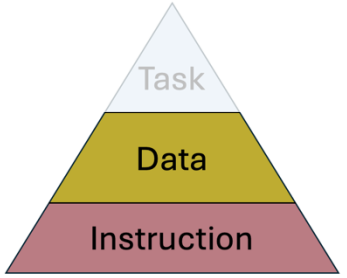


Security implications examples:

- Spectre 2018 (Branch speculation)
  - CPU Mitigations slow down execution
  - Compiler workarounds exist
  - Some CPU fixes can be disabled in firmware
- Meltdown 2017 (Cache side channel)

Searching for patterns in Conway's Game of Life





Instruction parallelism: 200 000x

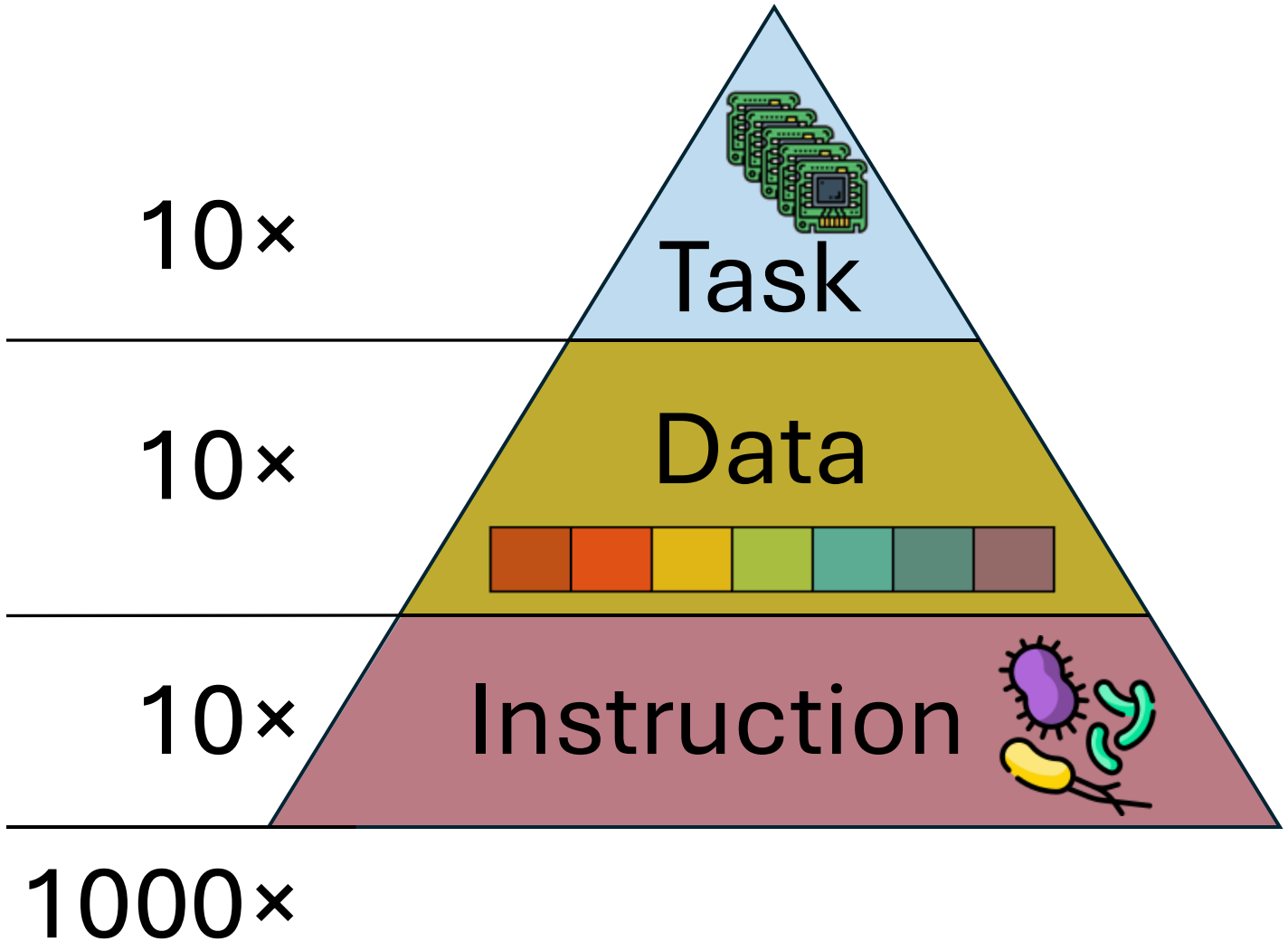
Data parallelism: 5 000x

Total speedup: 1 000 000 000x

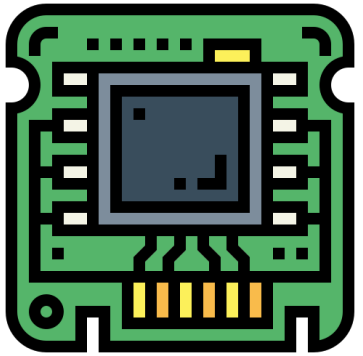
Algorithm	method	cells/gen/sec	speedup[4]
Naïve counting	count neighbor cells for every pixel	10^4	
JavaLife [35]	lookup table + linked list	$3 \cdot 10^7$	3 000 ×
QLife [67]	lookup table + quad tree	10^8	3 ×
LifeCube [62]	halfadder logic	$2 \cdot 10^9$	20 ×
Optimized LifeCube	low level assembly + stagger step + 64-ary tree	10^{10}	5 ×
GPU LifeCube	CUDA code running on a GPU	10^{13}	1 000 ×

Parallel hierarchy

Speedups multiply



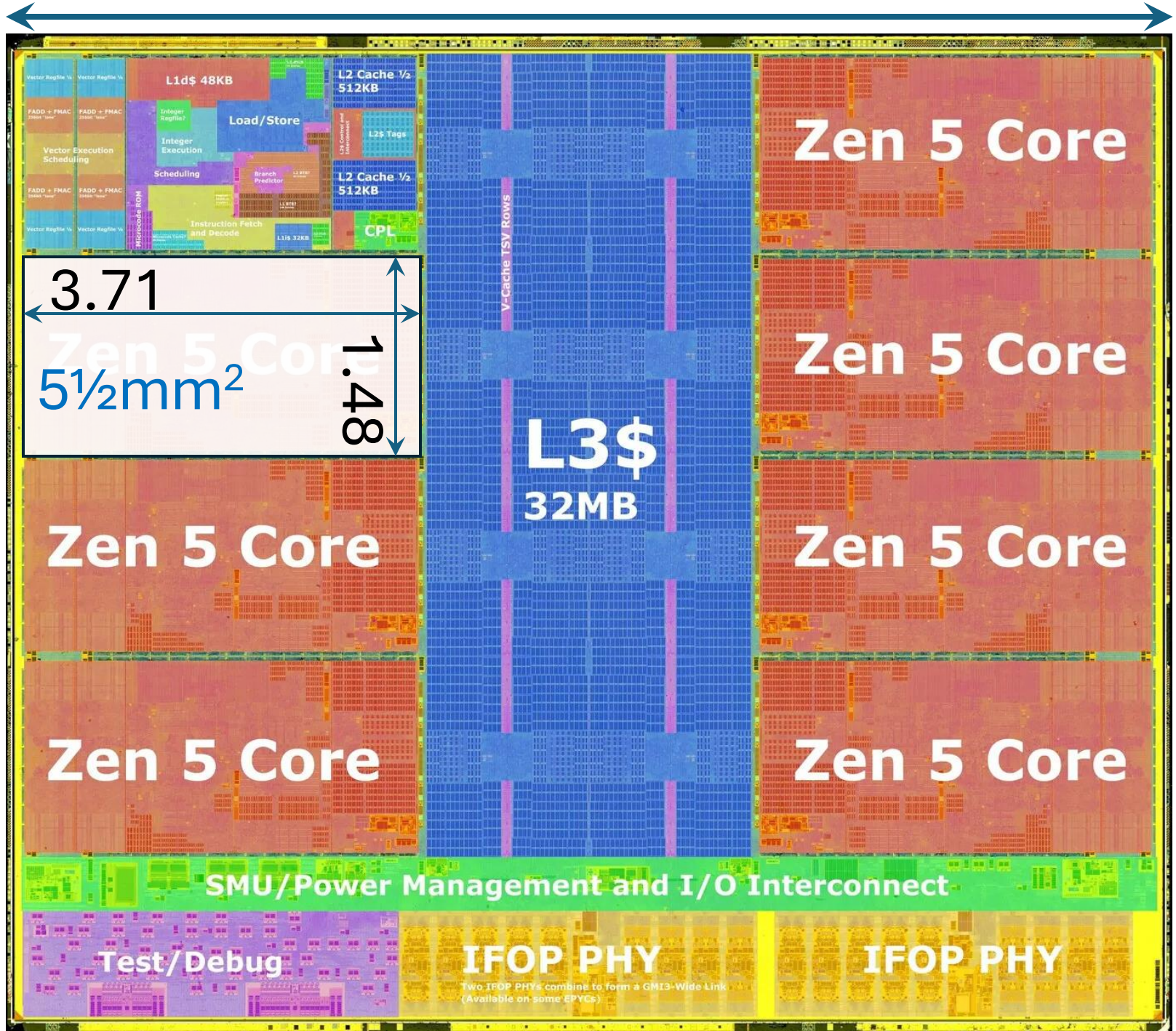
Intermezzo CPU vs GPU



- CPUs are low latency race cars 
  - \+ low latency
  - – low throughput
- GPUs are data parallel trucks 
  - – high latency
  - \+ high throughput
- With (superhuman) effort,
  latency can be exchanged for throughput

AMD Zen 5

9.07mm



1 of 8 cores

(2 threads) 5½mm² = 646 mio trans



GPU

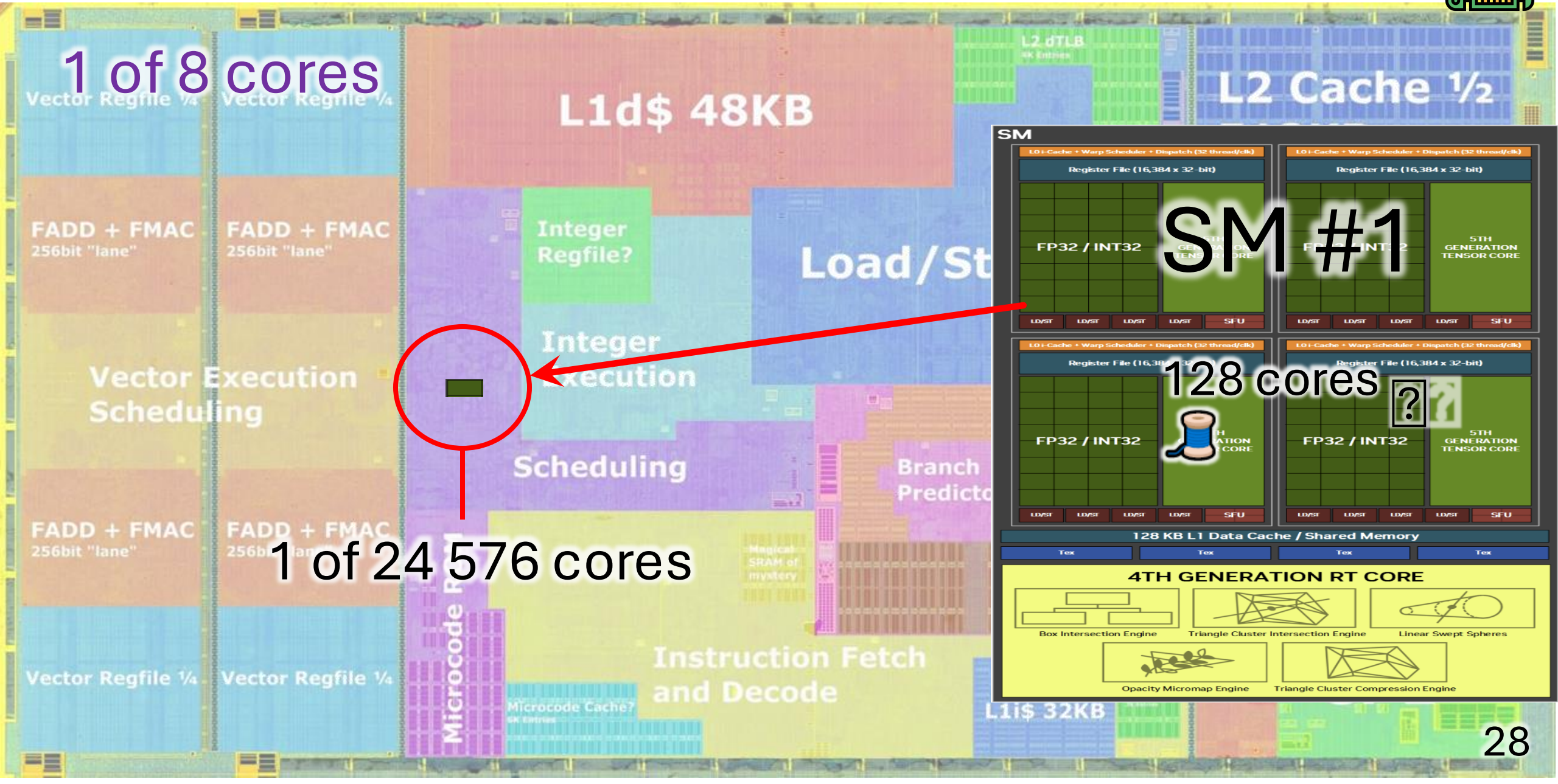
Nvidia
Blackwell
GB202

750mm²

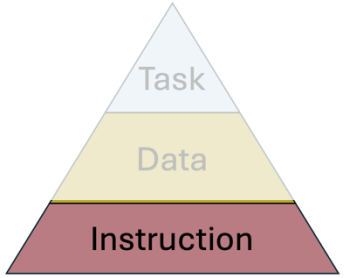
24 576



Zen 5 vs Blackwell core



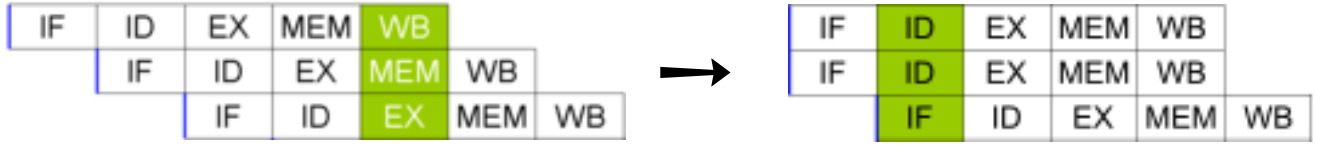
GPU vs CPU



- ~~pipelining~~



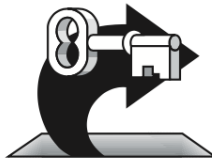
- ~~super scalar~~



- ~~branch prediction~~

- ~~cache prefetching~~ + ~~speculative execution~~

- ~~out-of-order execution~~ A B C → A C B



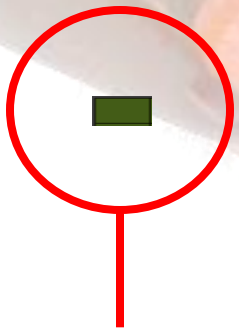
As is, GPU code ~100x slower than CPU

Why GPU slower?

Zen 5

5,5mm² per core = 1100x

646 m trans/core = 1175x



1 of 24 576 cores

0,005mm²

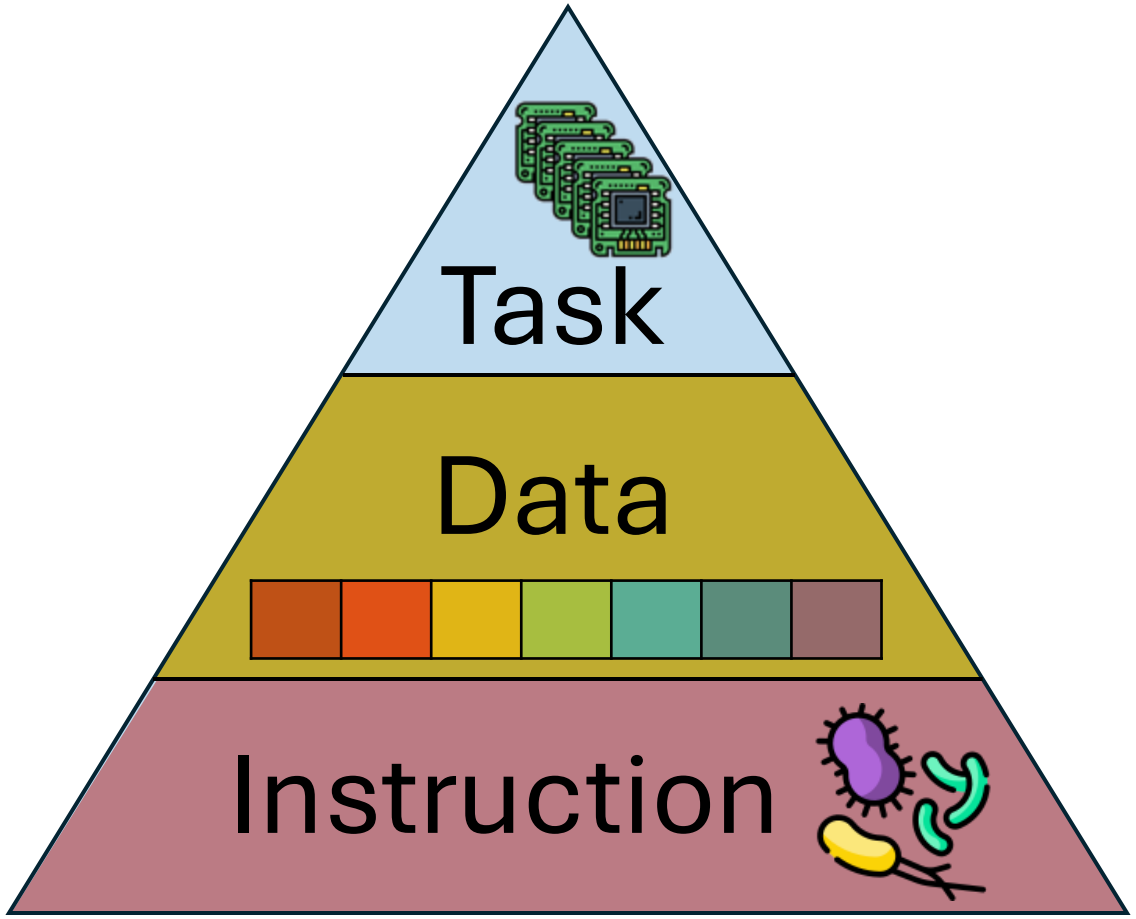
~550 k transistors

Parallel hierarchy…

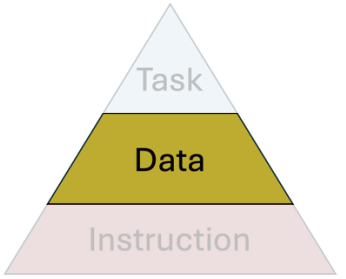
Task parallelism

Data parallelism

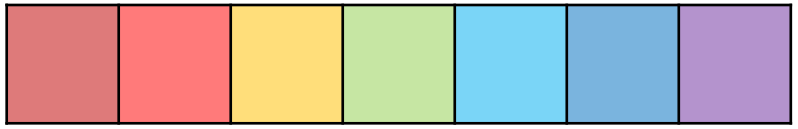
Instruction parallelism



Data parallelism (CPU)

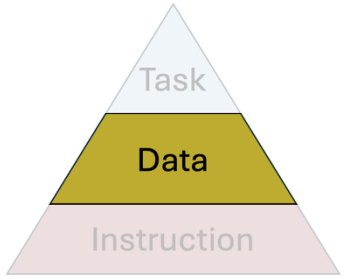


- Single instruction – multiple data
  - Vector instructions



  - Works on arrays of data `int A[16] + int B[16]`
    - Vectorization
      - Can be done by compiler
      - Or manual using framework
        C++: `#include <simd>`
        Java: `import jdk.incubator.vector.*;`
  - speed up:
    - more work per cycle
    - better memory efficiency
    - SIMD runs superscalar with other code

Vectorization



Example:

```
A, B: int32;
A += B;
```

On a 32-bit machine

```
mov b, [B]
add [A], b
```

32-bit regs
1 add
4 cycles

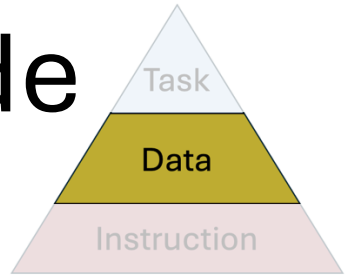
```
X, Y: int32[16];
X += Y;
```

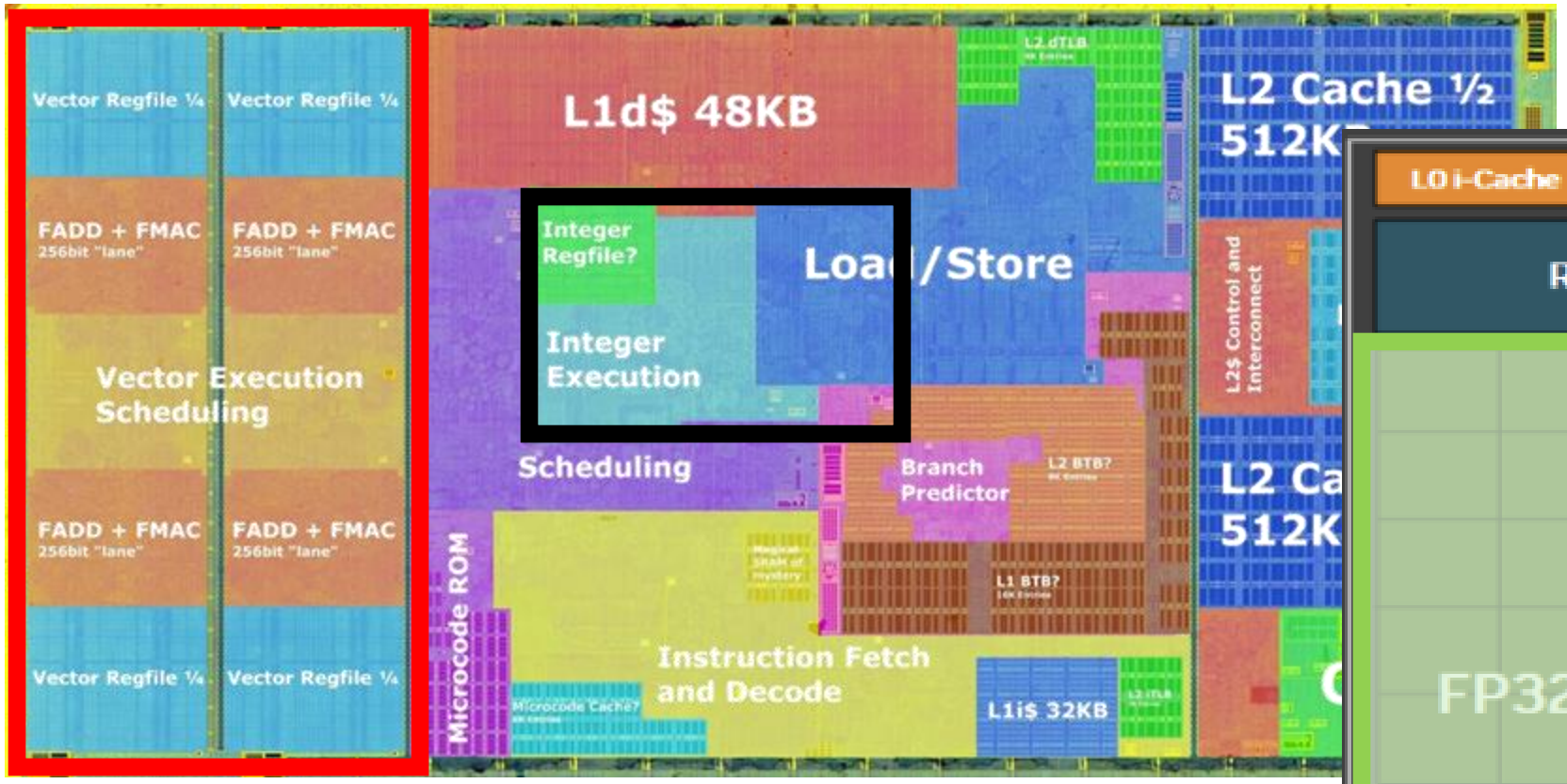
using vectorization

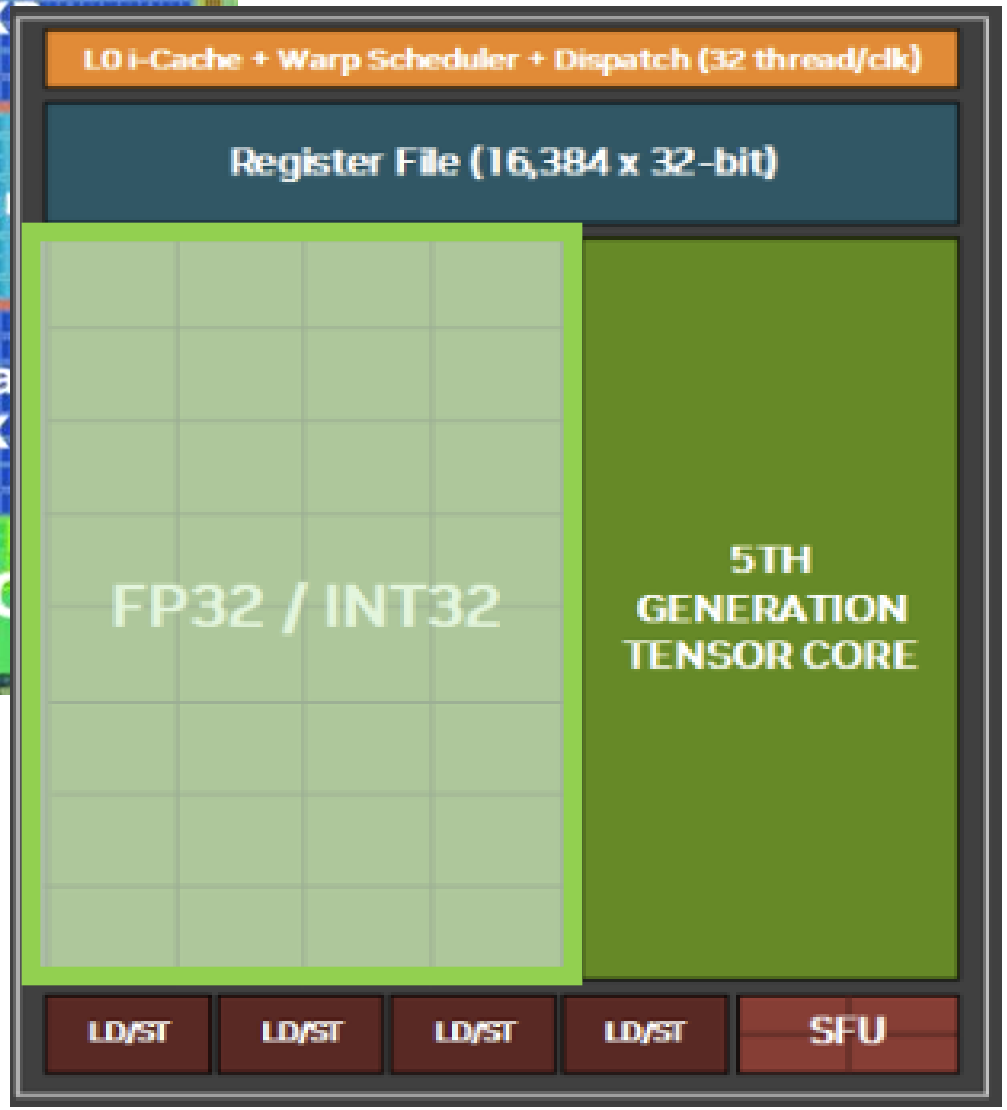
```
vmov32 bv, [Y]
vadd32 [X], bv
```

512-bit regs
16 adds
4 cycles

CPU: SIMD runs superscalar with other code

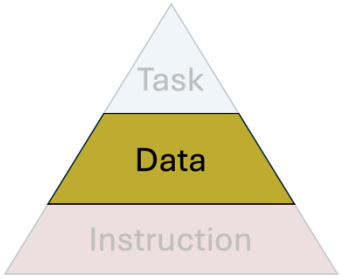






GPU: all code is SIMD

Data parallelism



Speeding up code using *manual** **vectorization**
is a mostly painless way to speed up your programs

If performance is critical,
and your algorithms are optimal

It is the easiest way to gain 2x-20x speedup

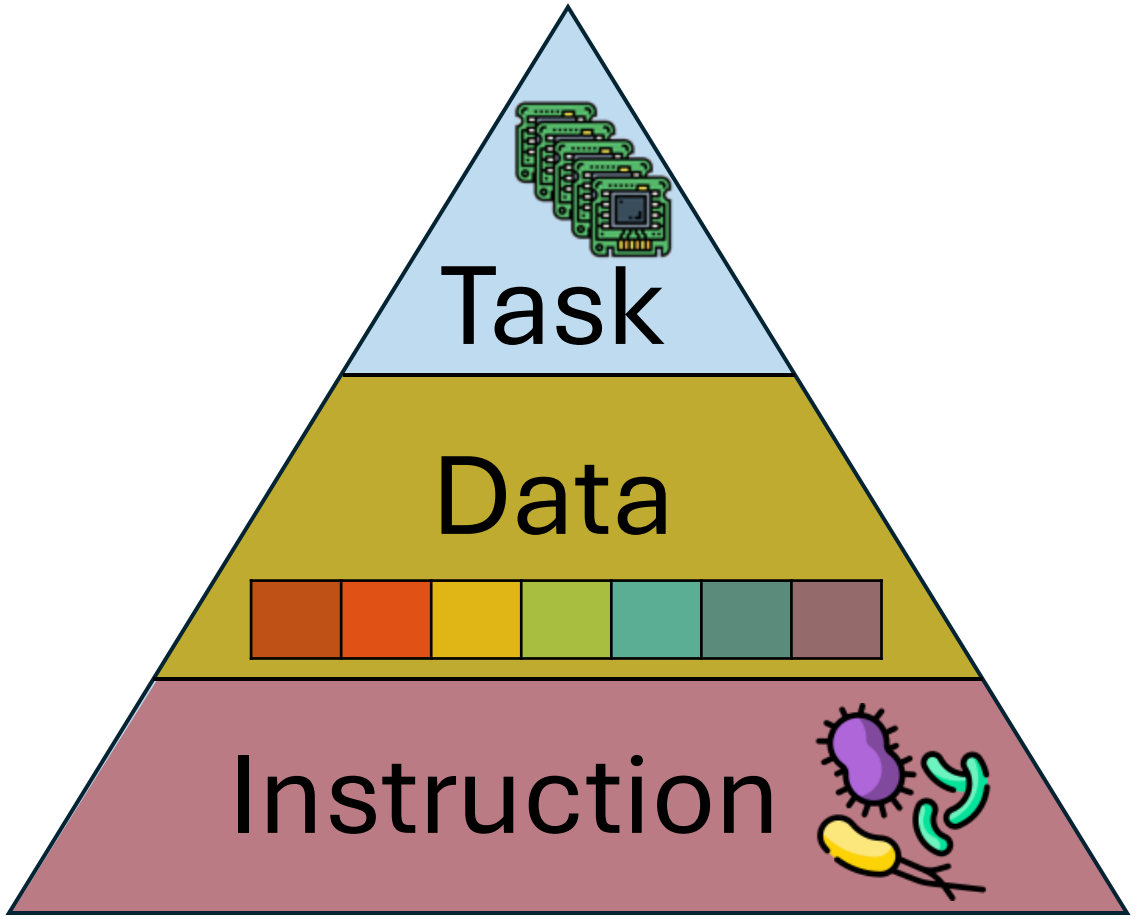
*) compilers suck at automatic vectorization

Parallel hierarchy…

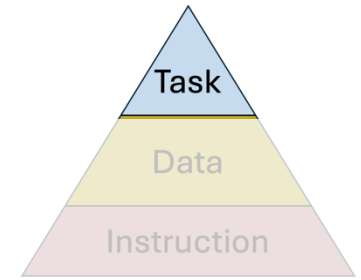
Task parallelism

Data parallelism

Instruction parallelism



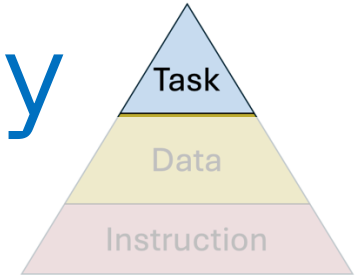
Task parallelism



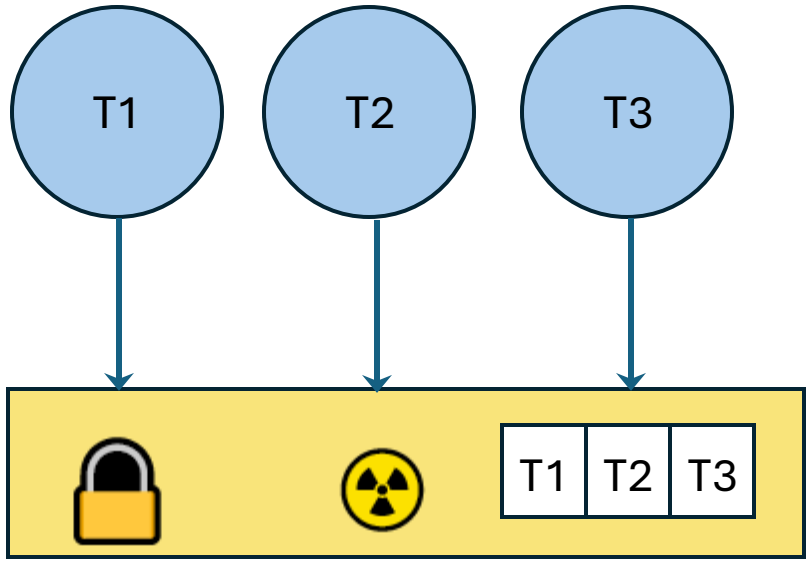
Tasks ≈ ‘Independent programs’

- Each task has its own data
  - Memory explosion
- **Communication** between tasks
  - Time consuming
  - Complex
  - Hard to prove **correctness**
  - No easy solution
  - Majority! of published research is incorrect

Distributed vs shared memory



Shared memory

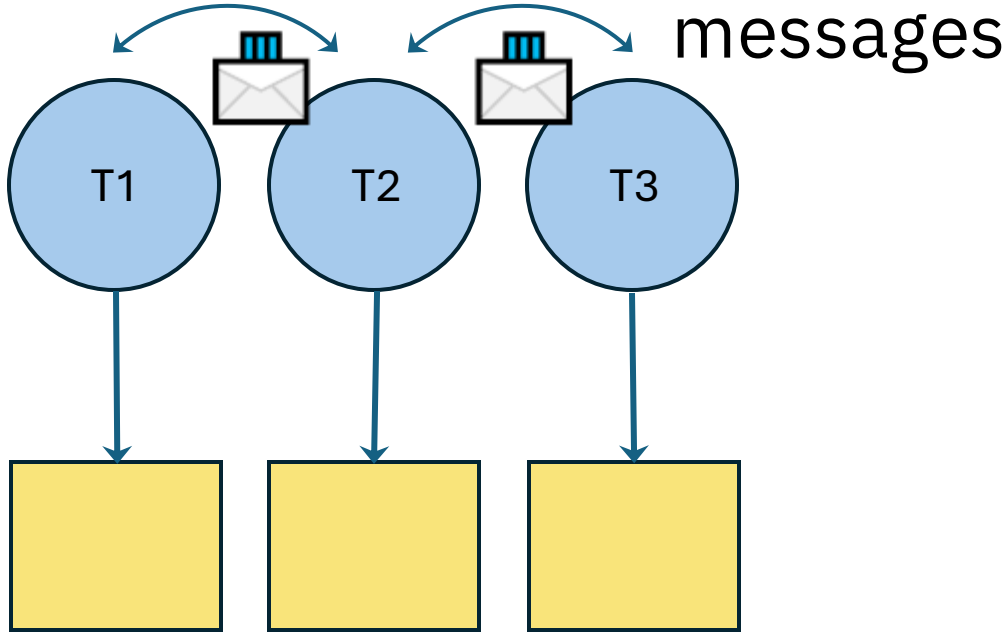


locks / atomics / partition

+ fast

– complex

Distributed memory

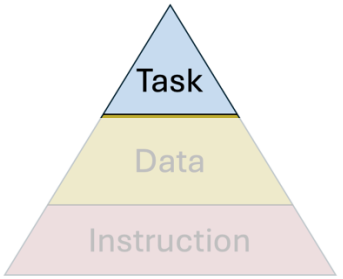


MPI: Message passing interface

– slow

+ provably correct

Shared memory tasks

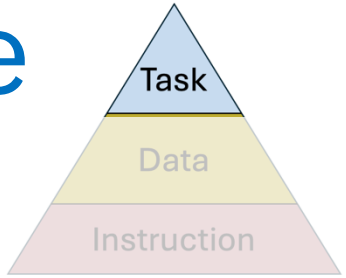


Communication happens using data structures

- queue in → | A | B | C | D | E | → out
- stack | A | B | C | D | E | ⇆ in/out
- tree

More complex structures = too much pain

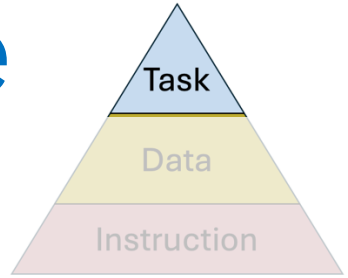
Locked/lock free/wait free



Shared memory access can be deconflicted

- **Locked**: one task locks, all other wait
  - \+ Easy to implement, efficient for locking task
  - – All other tasks must idle during lock
- **Lock free**: tasks only lock elements they need
  - \+ Other tasks can claim non-locked elements
  - – coordination extremely subtle and complex
- **Wait free**: lock free + other tasks can **help** locking task
  - \+ Reduces contention on ‘popular’ elements
  - – The most complex subject in all of CS

Locked/lock free/wait free



Why different mechanisms?

- **Locked**: one task locks, all other wait
  - if 2 tasks lock, program hangs = **deadlock**
  - if 1 task keeps locking, other tasks **starve**
- **Lock-free**: tasks only lock elements they need
  - \+ one task will always make progress
  - other tasks can still **starve** or **livelock**
- **Wait-free**: lock-free + other tasks can **help** locking task
  - \+ locking task will always succeed in *fixed number of steps*
  - \+ other tasks do not starve or livelock
  - high fixed overhead

Guarantees

Locked

- 1 thread can deadlock system

Lock free

- No one thread can deadlock system,
  but no guarantee on wait times

Wait free

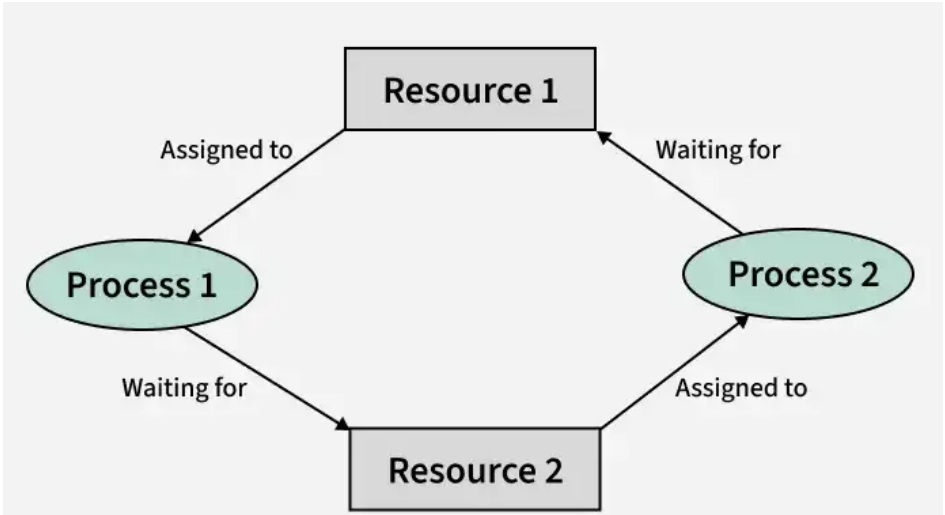
- No thread has to wait more than X steps (***steps***, not time)

Deadlock/starvation



Deadlock:

- task A locks A, needs B
- tasks B lock B, needs A

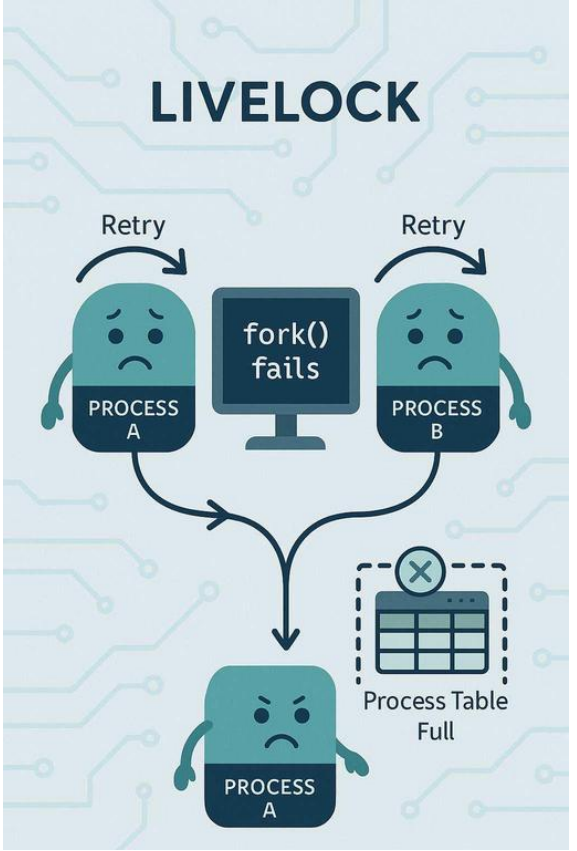


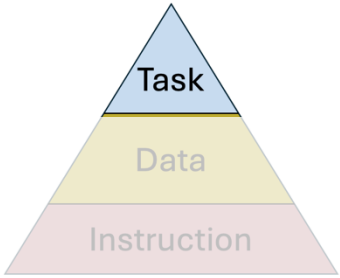
Starvation:

- NS task keeps winning
- EW task cannot get lock



Livelock





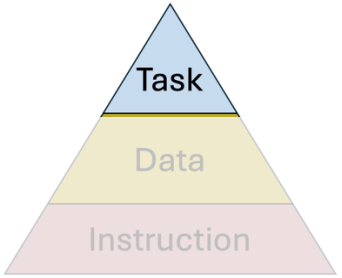
Livelock:

- no tasks wait
- no task can make progress

T1,T2:

```
try //livelock for T1,T2 if mem full
   while (memfree() < 100K) { wait(); }
   a = AllocateMem(100K) //locks
finally
   a.free()
```

How do locks work?



Out of scope for this course, but

- All processors have **special hardware** for locks
- This guarantees that only one task can get a lock
  - other tasks are made to wait
- processors also have support for micro-operations: **atomics**
  - T1, T2: `old = atomicAdd(x, 1);`
    - Before: x == 0, After: x == 2;
    - After: T1.old == 0 **or** 1, T2.old == 1 **or** 0, T1+T2.old == 1
    - never: T1.old == 1 **and** T2.old == 1
- Atomics can build lock-less parallel data structures
  - E.g.: queue, but **not** stack